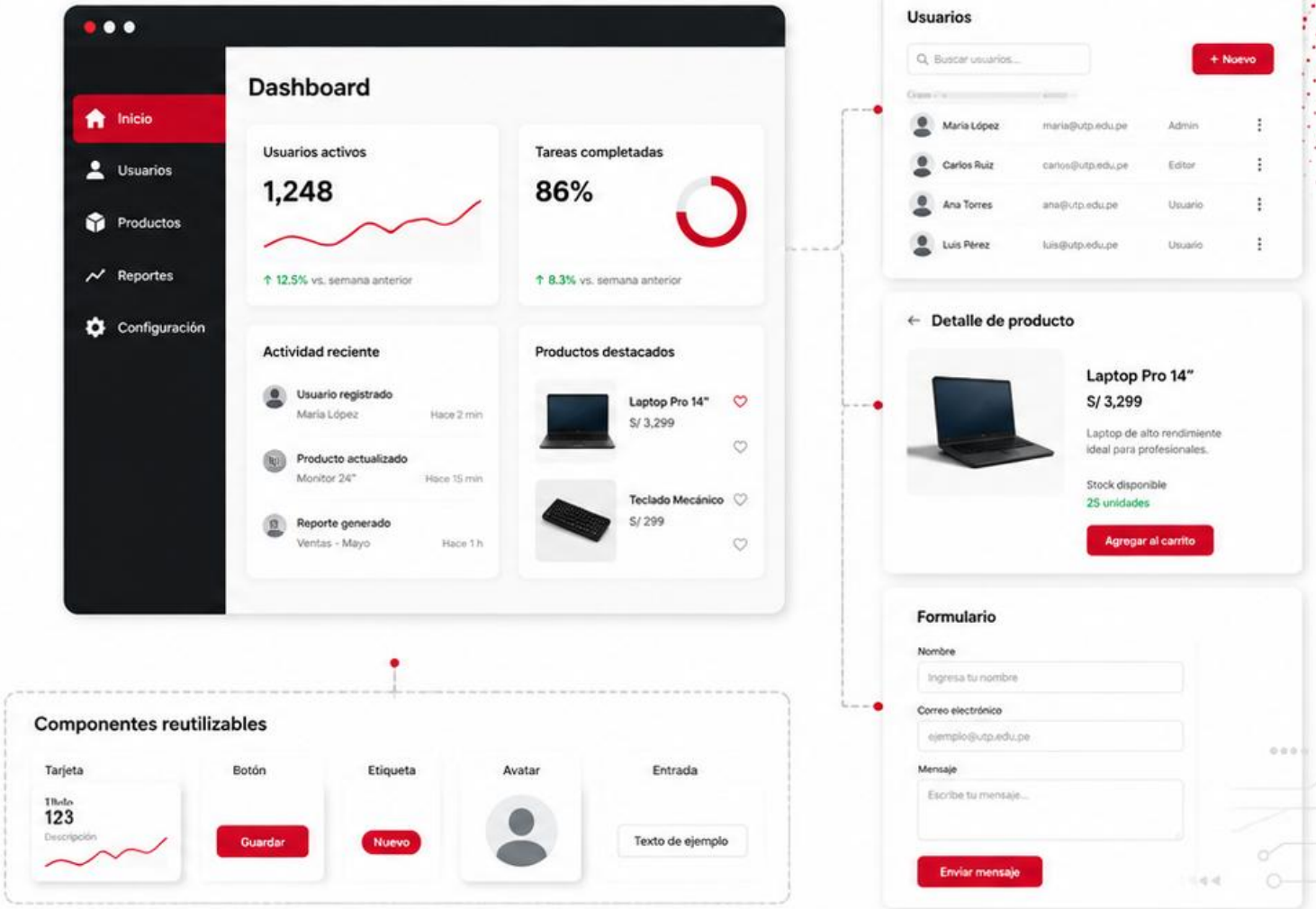
Universidad Tecnológica del Perú

Elementos de React

Curso: JavaScript Avanzado · Unidad 3: React · Semana 12

Docente: Mg. Ivan Eduardo Anchante Reyes.



Universidad Tecnológica del Perú

De una pantalla a una aplicación

Semana 11: proyecto React + JSX/TSX + estilos.
Semana 12: componentes + rutas + formularios → aplicación navegable.

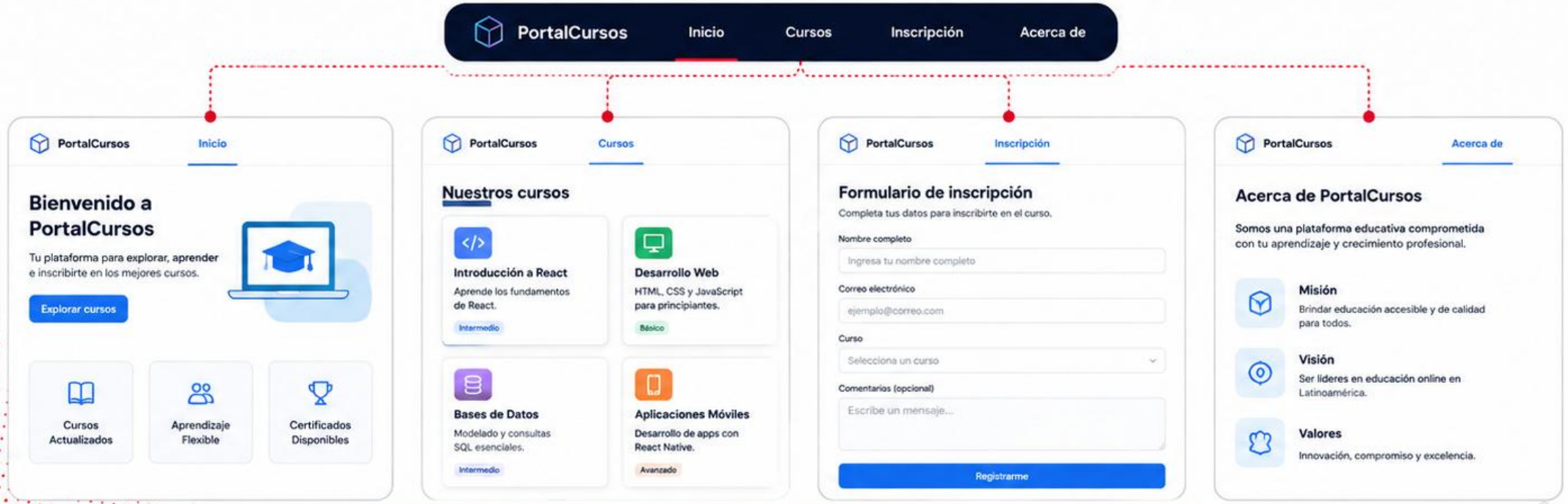


Pantalla única **Componentes** **Aplicación navegable**

Universidad Tecnológica del Perú

Reto de la semana

PortalCursos necesita **Inicio, Cursos, Inscripción** y **Acerca de.**
Debe **reutilizar tarjetas** y **registrar datos** de un formulario **sin usar hooks** todavía.



¿Cómo convertir una pantalla en una aplicación?

Universidad Tecnológica del Perú

Recuperamos la Semana 11

Proyecto React, JSX/TSX y estilos

? ¿qué renderiza **main.jsx**?

? ¿qué devuelve un **componente**?



? ¿qué ventaja **tiene JSX**?

? ¿dónde aplicaríamos **estilos**?

Universidad Tecnológica del Perú

Diagnóstico

¿Qué elemento de React resuelve cada necesidad?

Tres situaciones: una tarjeta se repite 12 veces; al pulsar “Cursos” cambia la URL; un usuario envía nombre y correo. Clasifica qué elemento resuelve cada necesidad.

1 Una tarjeta se repite muchas veces



Elemento de React:

2 Al pulsar “Cursos” cambia la URL



Elemento de React:

3 Un usuario envía nombre y correo



Elemento de React:

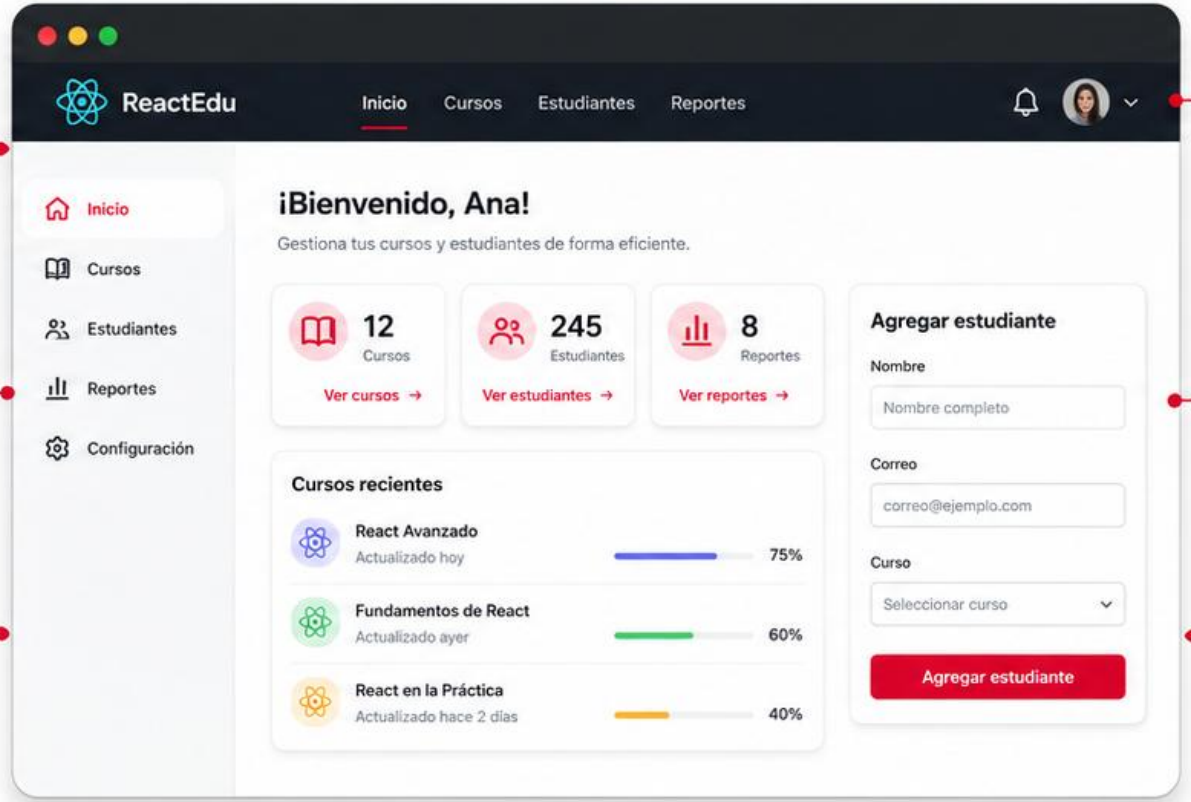
Universidad Tecnológica del Perú

¿Para qué sirve?

1. Mantener código
Facilita actualizar, corregir y dar mantenimiento a la aplicación.

2. Reutilizar UI
Los componentes se pueden reutilizar en distintas partes de la aplicación.

3. Separar responsabilidades
Cada componente tiene una función específica y clara.



4. Navegar sin recarga completa
Permite cambiar de vista de forma rápida y fluida.

5. Capturar datos
Los formularios permiten ingresar y validar información de los usuarios.

6. Escalar una aplicación
Una base sólida permite crecer y agregar nuevas funcionalidades.

Universidad Tecnológica del Perú

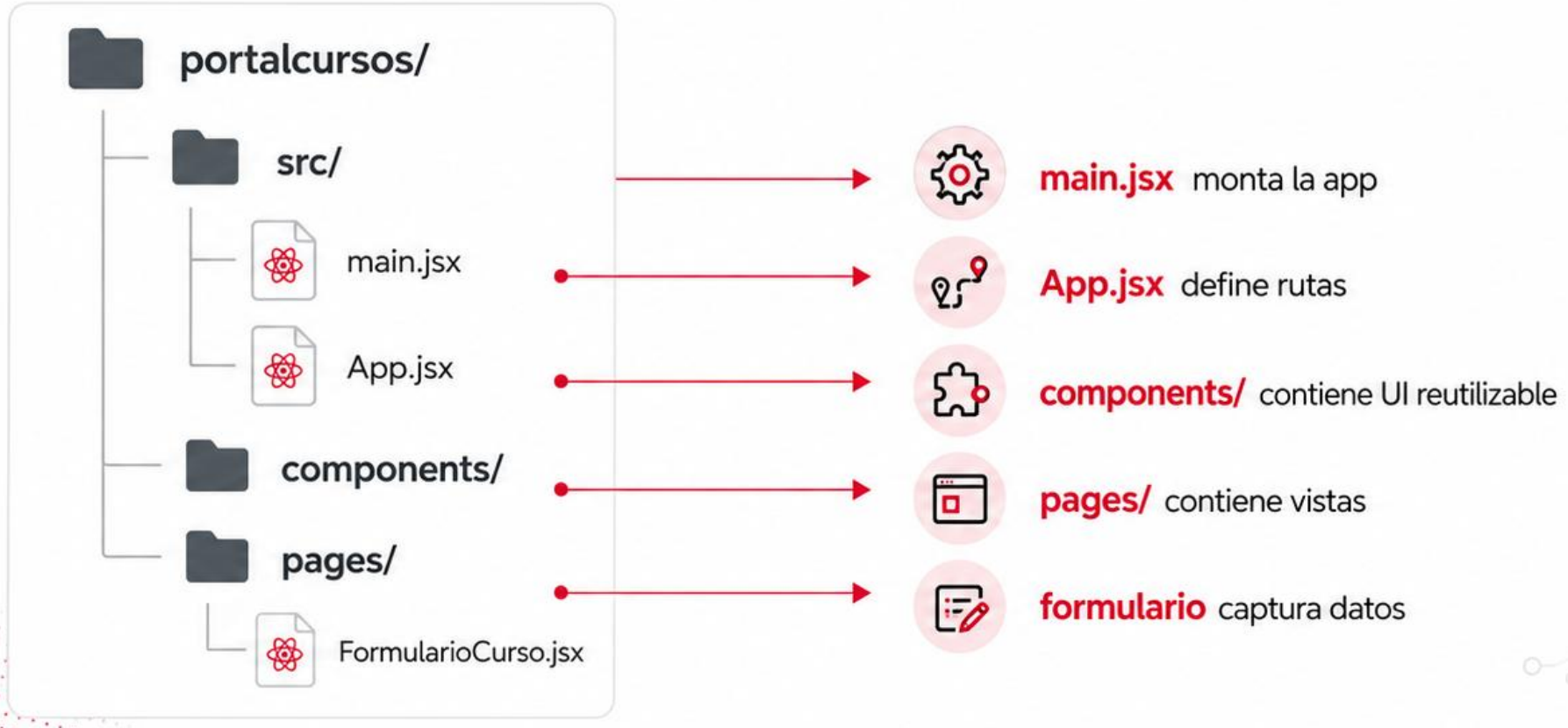
Logro de la sesión

Al finalizar la sesión, el estudiante implementa una aplicación React con navegación declarativa, componentes reutilizables y un formulario no controlado, organizando responsabilidades en archivos separados.



Mapa de la aplicación

PortalCursos



¿Qué es un componente?

Componente funcional: función que retorna JSX.

Debe tener nombre en PascalCase.

Ejemplo conceptual: CourseCard.

UI (Vista)

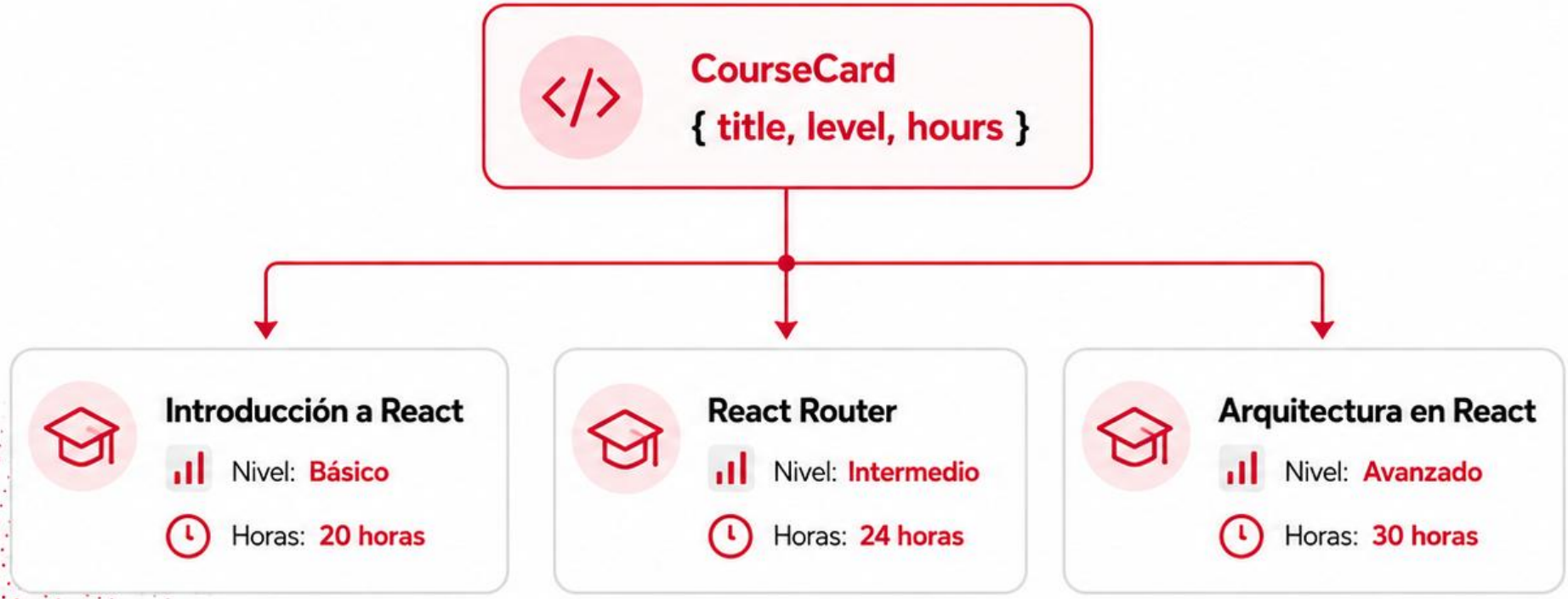


Código (Componente)

```jsx
function CourseCard() {
  return (
    <div className="course-card">
      <h3>React Básico</h3>
      <p>Aprende los fundamentos de React</p>
      <button>Ver curso</button>
    </div>
  );
}
```

Props

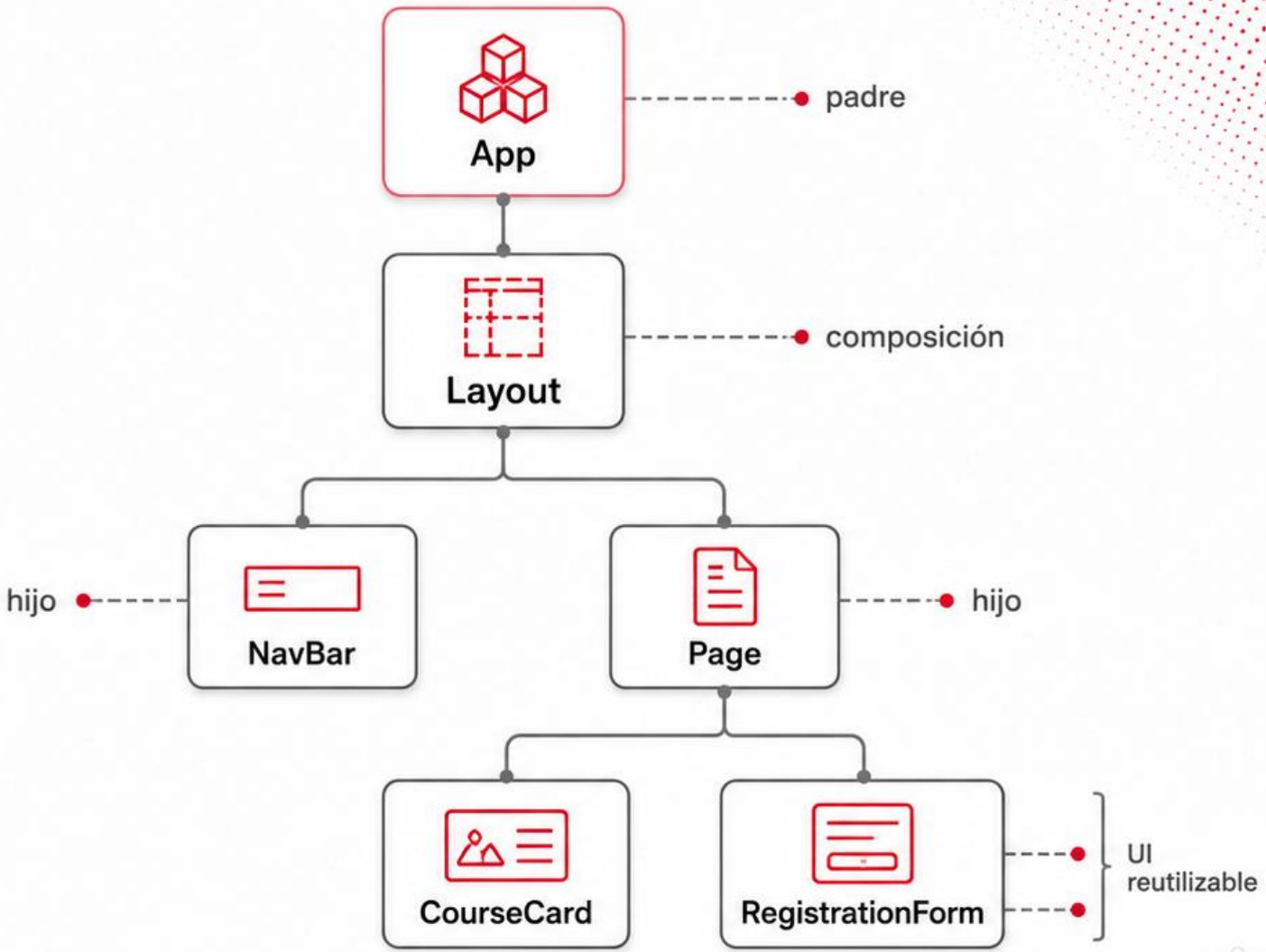
- **CourseCard** recibe **title, level** y **hours**.
- Tres tarjetas usan la misma estructura con valores distintos.



Composición de componentes

Cada nivel compone hijos

App → Layout → NavBar + Page → CourseCard / RegistrationForm



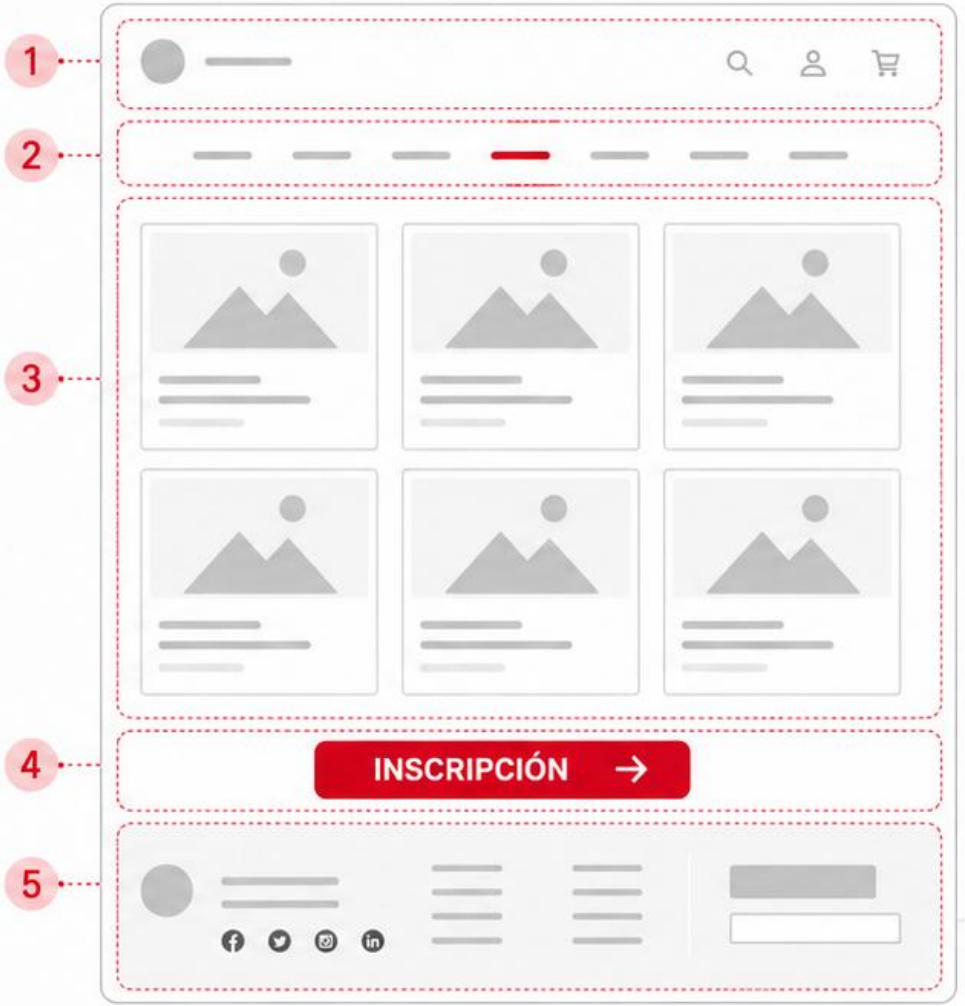
Microejercicio: divide la interfaz

Selecciona 2–4 componentes y justifica

Mockup de catálogo: encabezado, menú, 6 tarjetas, botón de inscripción y pie.

¿Qué componentes crearías?

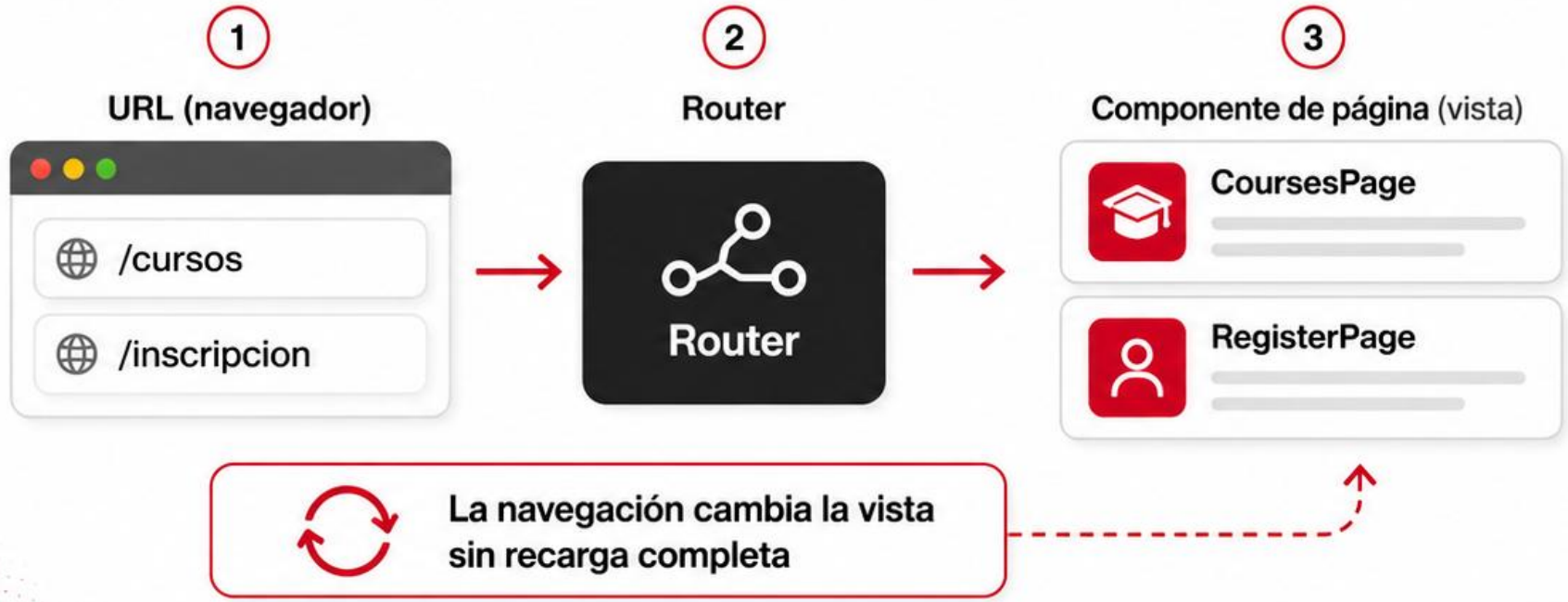
Actividad visual, sin solución visible



Universidad Tecnológica del Perú

Navegación en una SPA

URL ⟶ router ⟶ componente de página



- **Ruta "/cursos"** → CoursesPage; "/inscripcion" → RegisterPage.
  La navegación cambia la vista sin recarga completa.

BrowserRouter

main.jsx envuelve <App />

Usaremos react-router-dom por continuidad del proyecto; el paquete actual reexporta React Router.

```
import { BrowserRouter } from "react-router-dom";
import App from "./App";

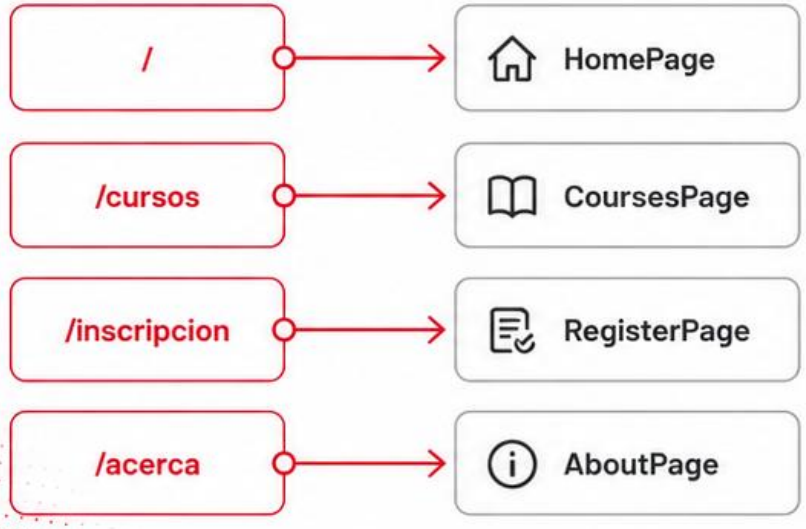
<BrowserRouter>
  <App />
</BrowserRouter>
```

main.jsx envuelve la aplicación para habilitar la navegación.

Routes y Route

<Routes> agrupa reglas

Añadir /, /cursos, /inscripcion y /acerca.




```jsx
<Routes>
  <Route path="/" element={<HomePage />} />
  <Route path="/cursos" element={<CoursesPage />} />
  <Route path="/inscripcion" element={<RegisterPage />} />
  <Route path="/acerca" element={<AboutPage />} />
</Routes>
```

Cada **Route** vincula una URL con una vista

Link y NavLink

Navegación declarativa y enlace activo

NavLink permite marcar el enlace activo; Link sirve para navegación declarativa. Menú: Inicio · Cursos · Inscripción · Acerca de.

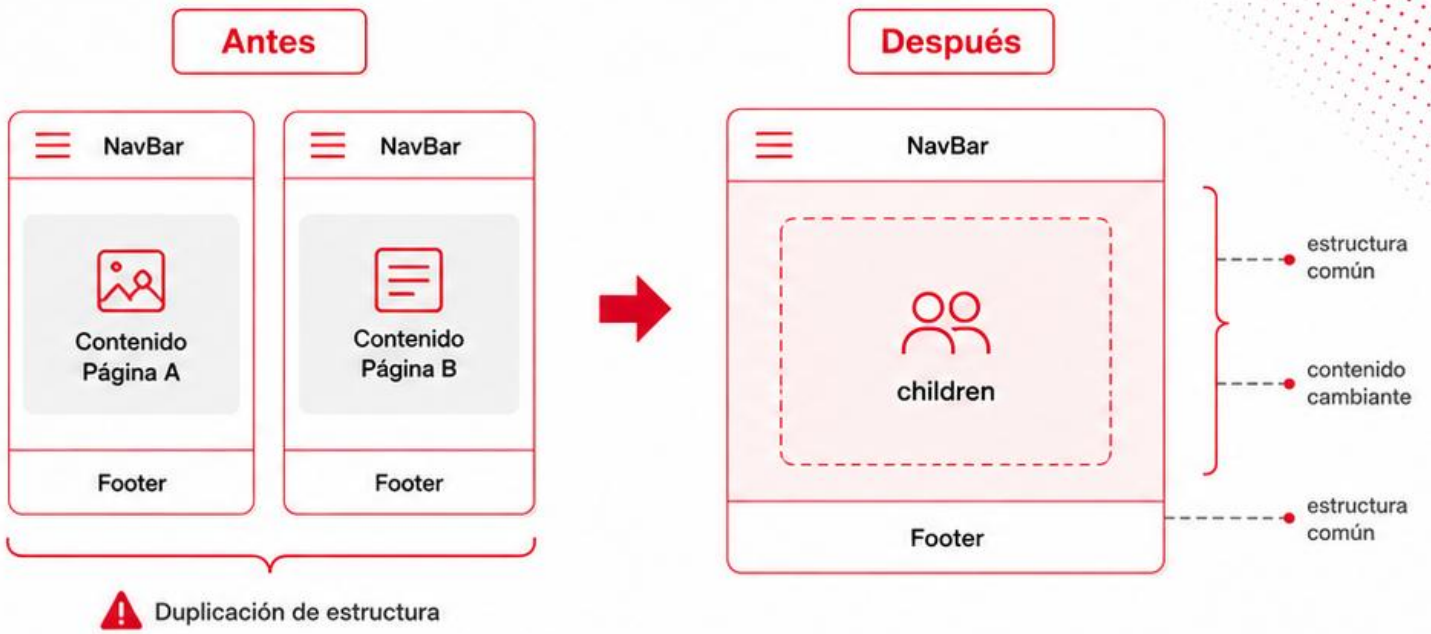



```
<nav>
    <Link to="/">Inicio</Link>
    <NavLink to="/cursos">Cursos</NavLink>
    <NavLink to="/inscripcion">Inscripción</NavLink>
    <NavLink to="/acerca">Acerca de</NavLink>
</nav>
```

Layout común

Una estructura fija con contenido cambiante.

Layout contiene **NavBar**, **main** y **footer**; recibe **children**. **App.jsx** envuelve cada página con Layout o mantiene estructura común.




```
function Layout({ children }) {
  return (
    <>
      <NavBar />
      <main>{children}</main>
      <Footer />
    </>
  );
}
```

Ruta 404

Una respuesta clara para rutas inexistentes.

- **Route path="*"** muestra **NotFoundPage** con enlace para volver al inicio.



JSX

```jsx
<Route path="*" element={<NotFoundPage /> />
```

Universidad Tecnológica del Perú

Demo: PortalCursos navegable

Comprobamos rutas, URL y enlace activo.

Abrir Inicio, Cursos, Inscripción, Acerca de y una URL desconocida. Observar URL, NavLink activo y página renderizada.



URL
Observar la URL en la barra del navegador.

NavLink activo
Verificar qué enlace de navegación está activo (resaltado).

Página renderizada
Confirmar el contenido mostrado en cada ruta.

Formularios sin adelantar hooks

Esta semana: formulario no controlado

Usaremos <form onSubmit>, inputs con name, e.preventDefault() y new FormData(form).
No usamos useState todavía; state Hooks se desarrollan en Semana 13.

Semana 12 ✓ Formulario no controlado (HOY)

Formulario no controlado

Nombre

name="nombre"

Correo

name="correo"

Enviar

onSubmit

e.preventDefault()

new FormData(form)

```
<form onSubmit={handleSubmit}>
  <input name="nombre" />
  <input name="correo" />
</form>

function handleSubmit(e) {
  e.preventDefault();
  const form = e.currentTarget;
  const data = new FormData(form);
}
```

Semana 13 Formulario controlado con useState

Formulario controlado

Nombre

value={nombre}

Correo

value={correo}

Enviar

useState()

Actualizar estado

Render reactivo

Todavía no

Se desarrolla en Semana 13.

Campos y atributo name

Cada campo necesita una clave para formar los datos

FORMULARIO DE INSCRIPCIÓN

Nombre completo *

María Fernanda López — name="fullName"

Correo electrónico *

maria.lopez@utp.edu.pe — name="email"

Curso *

Desarrollo Web — name="course"

ENVIAR INSCRIPCIÓN

OBJETO RESULTANTE (FormData)

```
{
  "fullName": "María Fernanda López",
  "email": "maria.lopez@utp.edu.pe",
  "course": "Desarrollo Web"
}
```

```
FormData.get("email")
→ "maria.lopez@utp.edu.pe"
```

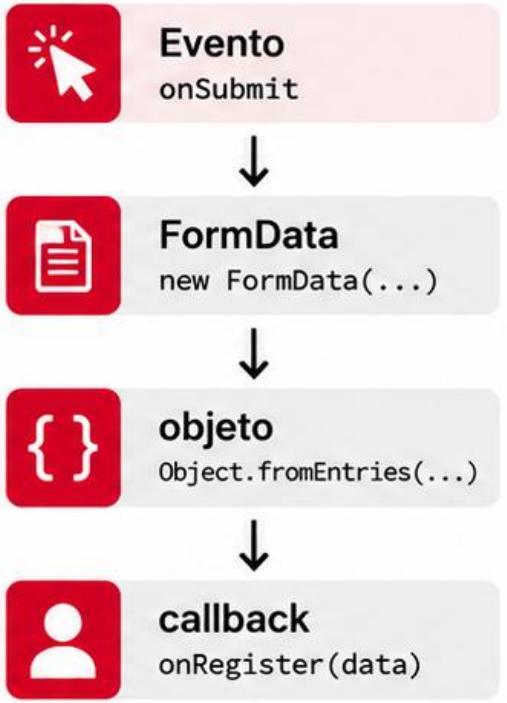
Obtiene el valor asociado a la clave "email".

Universidad Tecnológica del Perú

onSubmit + preventDefault + FormData

Del evento al objeto de datos

```
function handleSubmit(e) {
  e.preventDefault();
  const formData = new FormData(e.currentTarget);
  const data = Object fromEntries(formData entries());
  onRegister(data);
  e.currentTarget reset();
}
```



Formulario como componente

El padre decide qué hacer con los datos

RegisterPage (Padre)

Define qué hacer con los datos y pasa la función como prop.

```
function RegisterPage() {
  const onRegister = (data) => {
    // Aquí decide qué hacer
    console.log('Datos:', data);
    // Ejemplo: enviar a API
  };

  return (
    <RegistrationForm onRegister={onRegister} />
  );
}
```

prop
onRegister

callback
callback(data)

RegistrationForm (Hijo)

Recibe **onRegister** como prop y lo invoca cuando el usuario envía el formulario.

```
function RegistrationForm({ onRegister }) {
  const handleSubmit = (e) => {
    e.preventDefault();
    const data = { name, email, password };
    onRegister(data); // callback al padre
  };

  return ( /* formulario */ );
}
```

Validación HTML útil

La validación nativa mejora la experiencia del usuario

- Usar **type="email", required, minLength** y **select required** cuando aplique.
- Esta validación **mejora UX**, pero **no reemplaza validación de servidor** en sistemas reales.

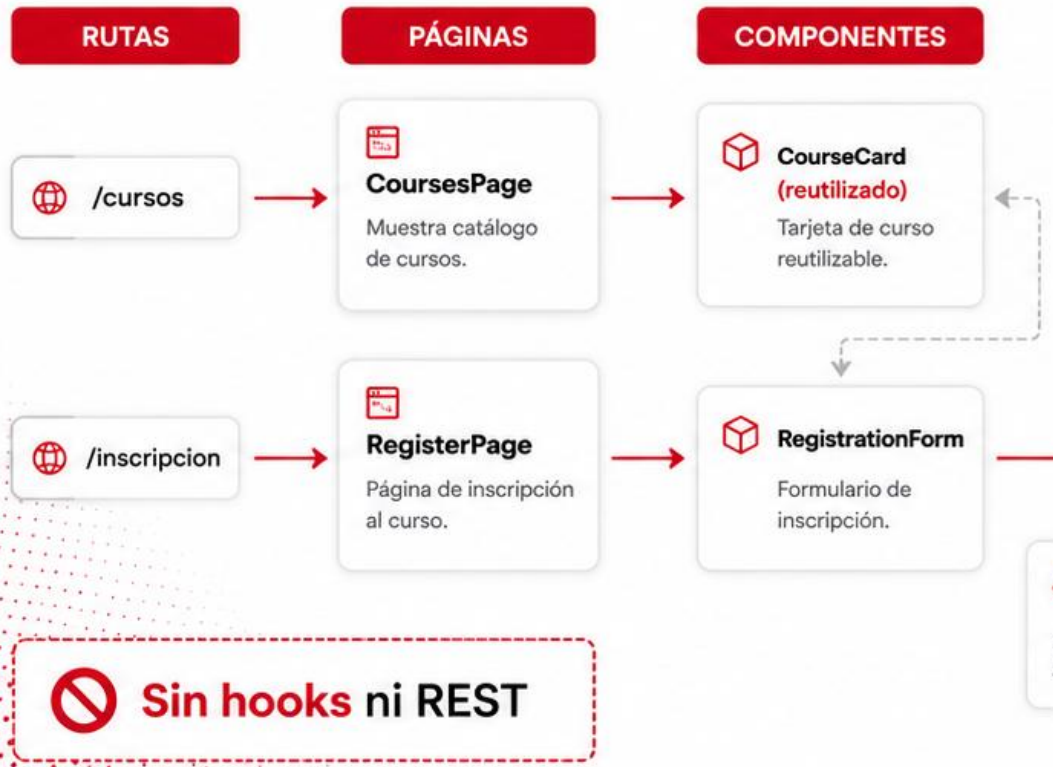


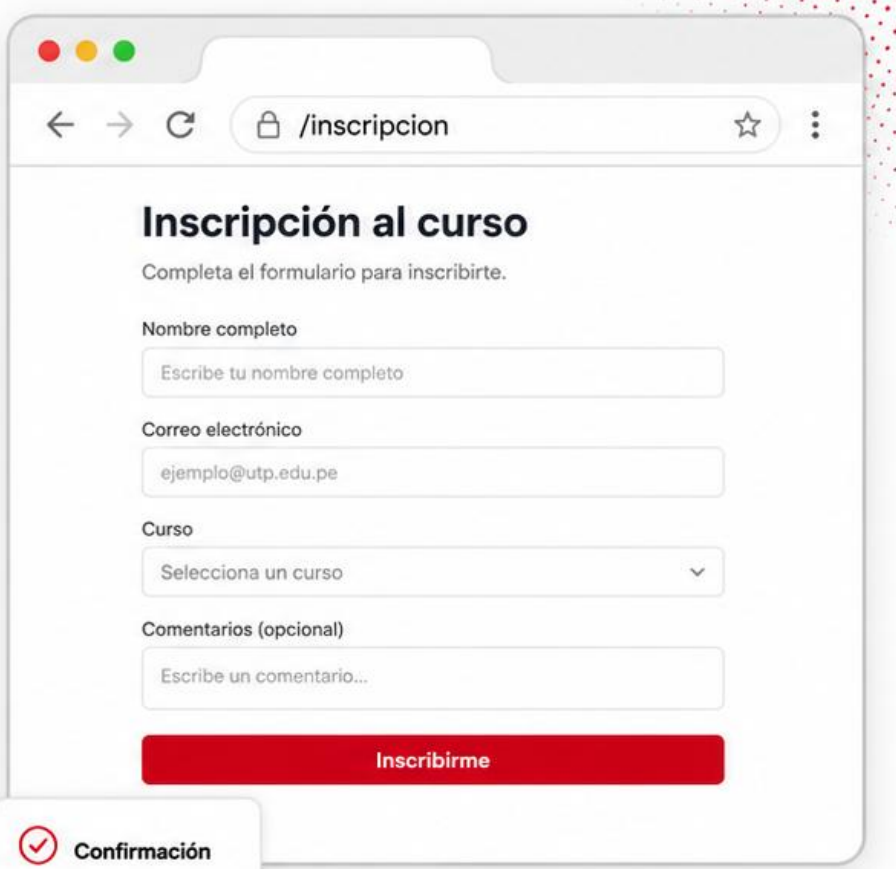
Mejora UX, pero no reemplaza validación de servidor en sistemas reales.

Universidad Tecnológica del Perú

Integración PortalCursos

Rutas, páginas y componentes trabajando juntos





Microdebug

Diagnostica el problema, explica el efecto y corrige.

Identifica el efecto y corrige.

A)

```
<NavLink to="cursos">Cursos</NavLink>
```

desde una ruta inesperada

Efecto:

Corrección:

B)

```
<input type="text" />
```

sin name

Efecto:

Corrección:

C)

```
<a href="/cursos">Cursos</a>
```

para ruta interna

Efecto:

Corrección:

Universidad Tecnológica del Perú

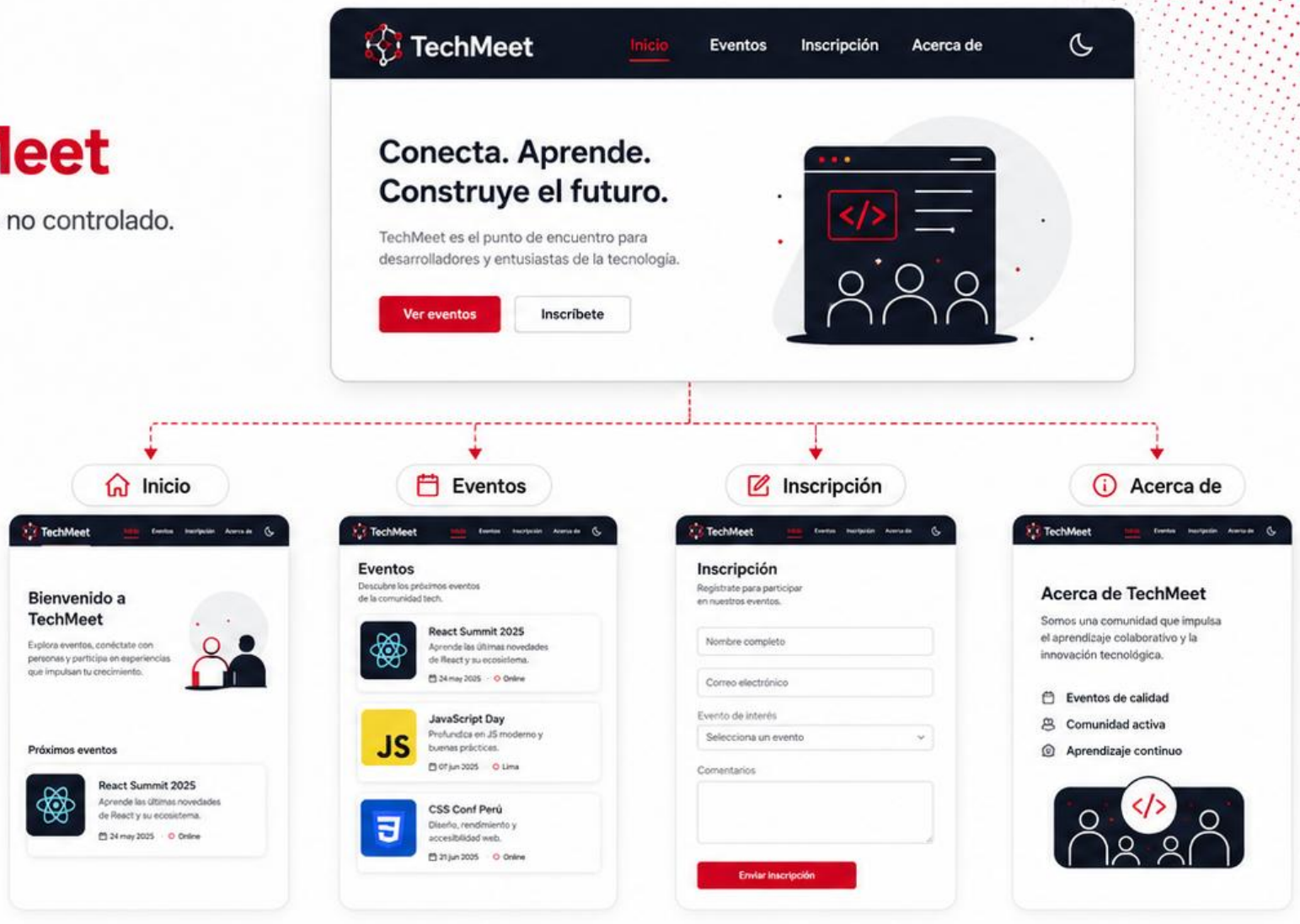
Reto individual: TechMeet

Aplicación React con rutas, componentes y formulario no controlado.

INDIVIDUAL · 50 min.

TechMeet necesita Inicio, Eventos, Inscripción y Acerca de; tarjetas reutilizables y formulario no controlado.

Entregable: aplicación navegable con vistas y registro de datos.



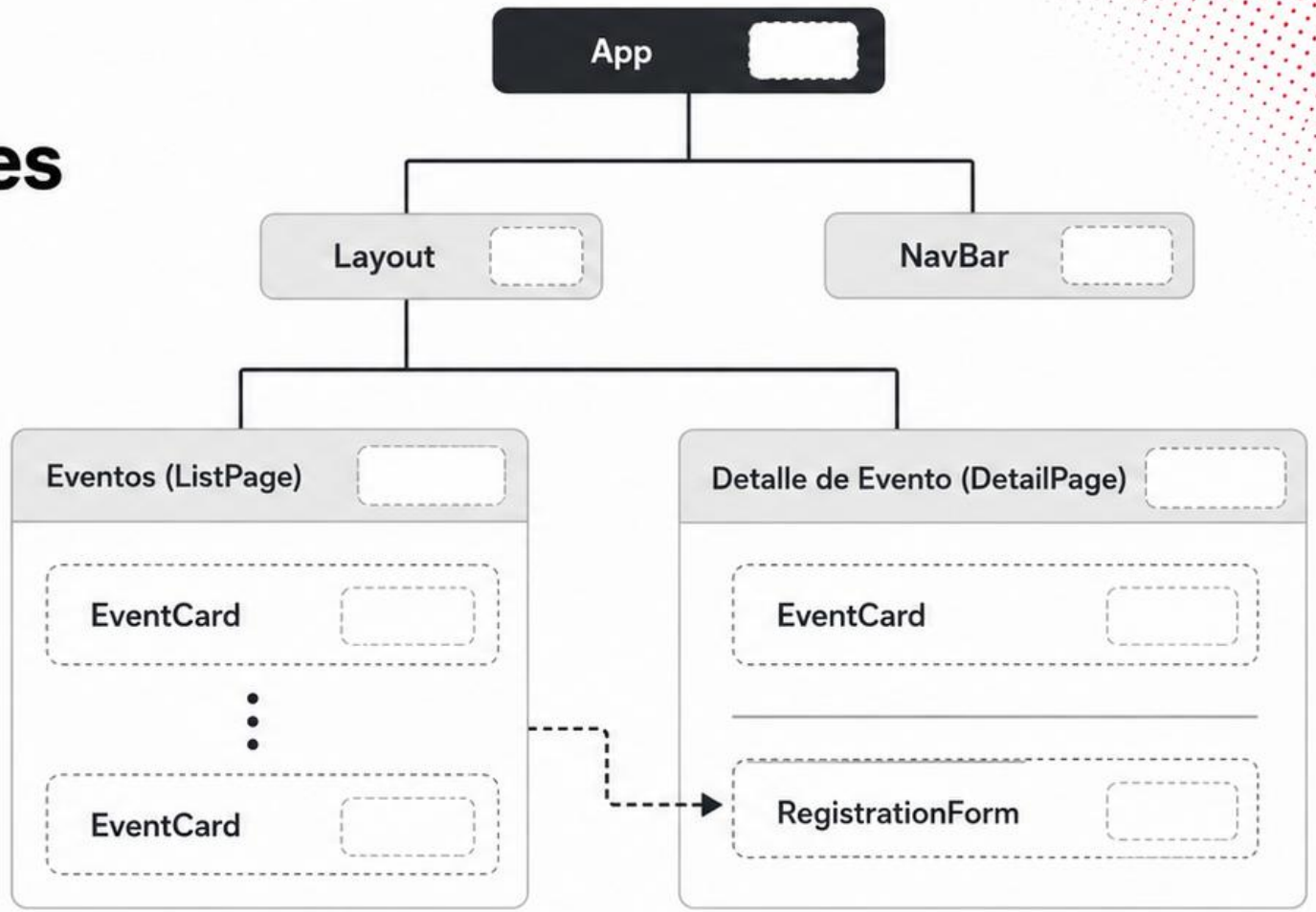
Parte A — Componentes

TechMeet: estructura y responsabilidades

- Crear/Completar NavBar, EventCard y RegistrationForm.
- EventCard recibe props. RegistrationForm recibe onRegister.

Criterios

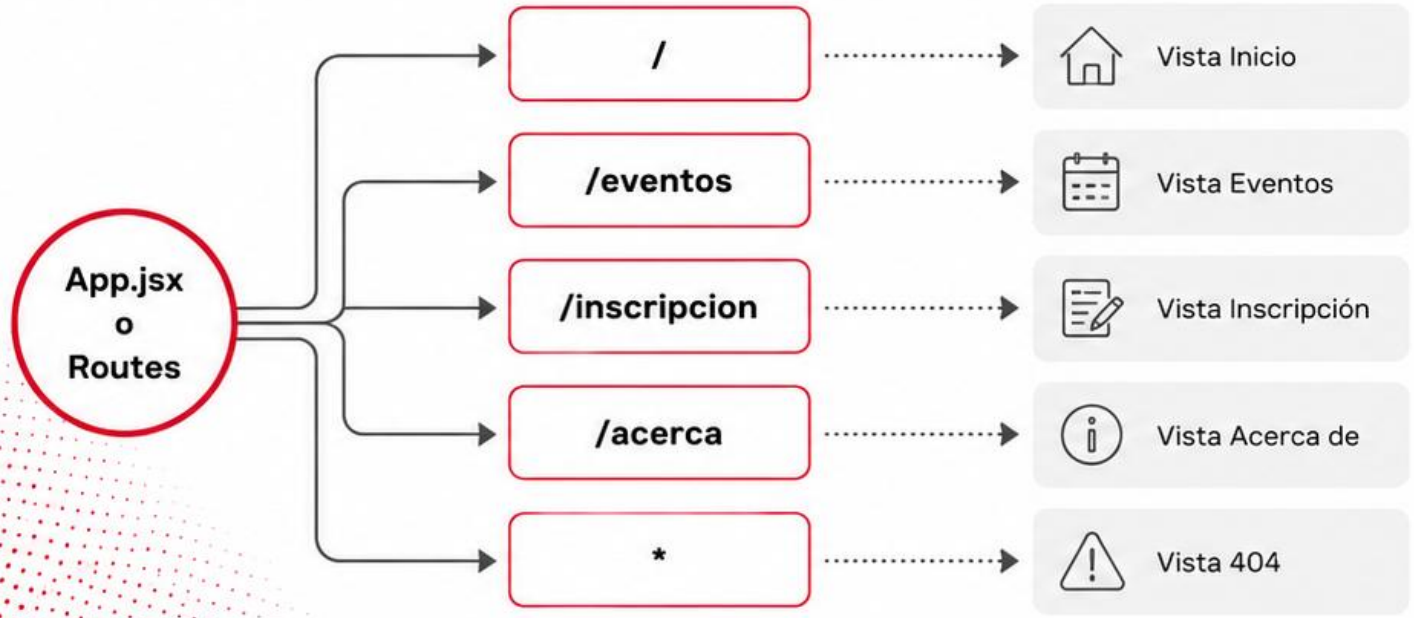
- Responsabilidad única
- Props bien definidas
- Reutilización



Parte B — Navegación

Rutas y menú con NavLink

- Implementar /, /eventos, /inscripcion, /acerca y *.
- Crear menú con NavLink; la URL y vista deben cambiar correctamente.

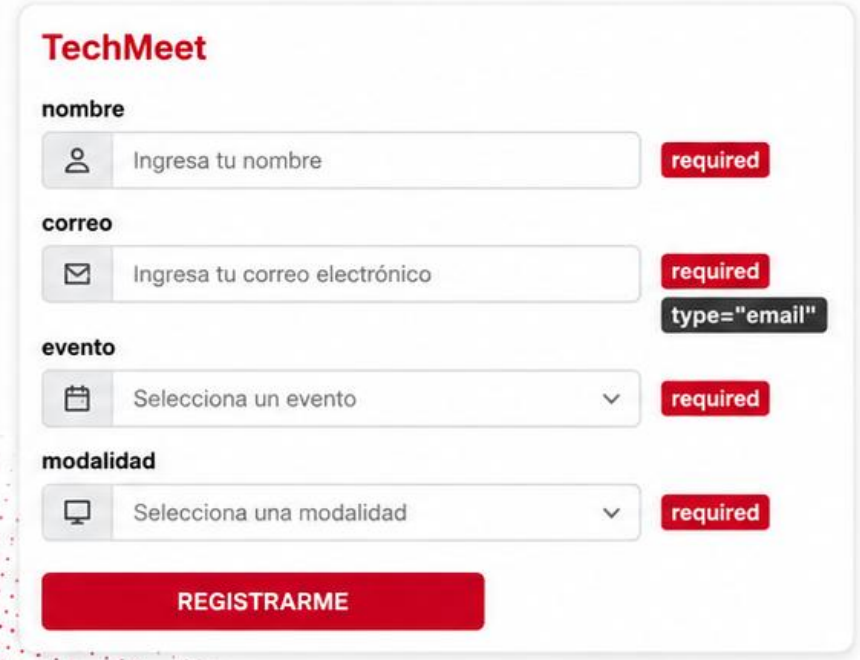


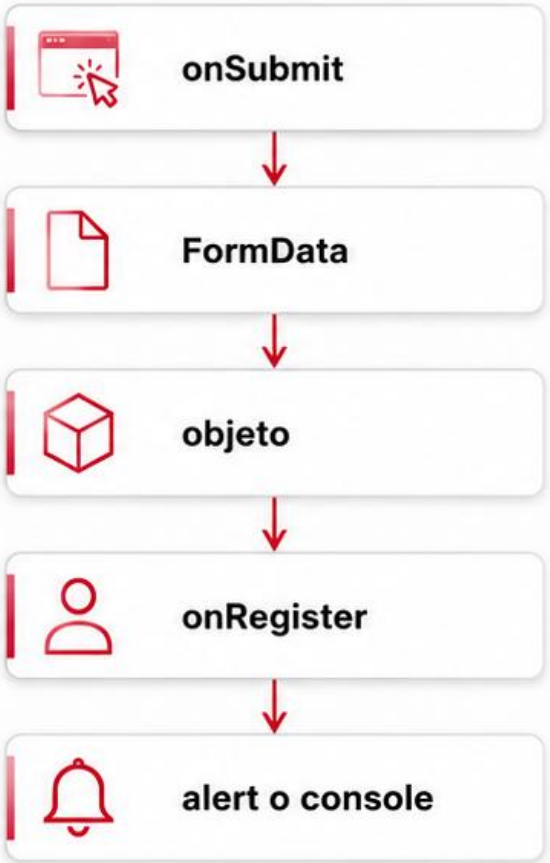
- URL correcta
- Vista renderizada
- NavLink activo
- Ruta 404

Parte C — Formulario

Registro en TechMeet sin adelantar hooks

Campos: nombre, correo, evento y modalidad. **required;** email **type="email".** onSubmit evita recarga, construye objeto y llama onRegister. Mostrar confirmación mediante alert o console.



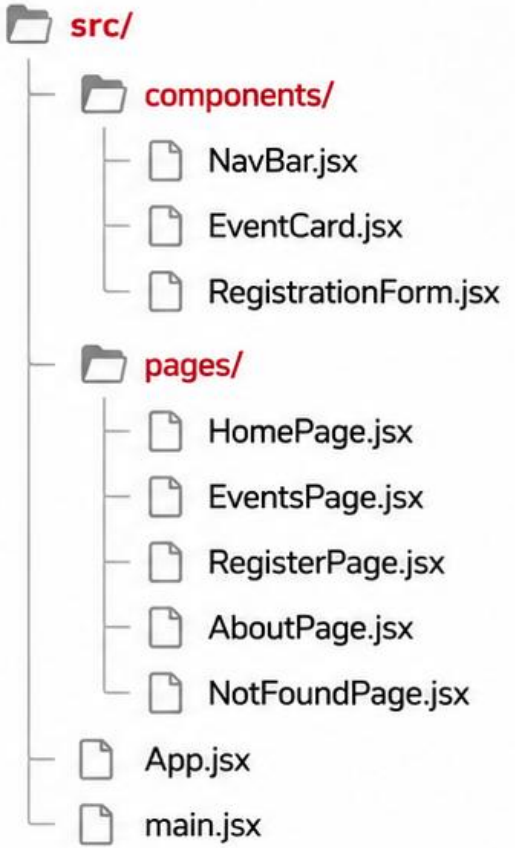


Parte D – Integración y prueba

Comprobar antes de entregar

- [x] Comprobar: componentes separados
- [x] Comprobar: rutas
- [x] Comprobar: NavLink activo
- [x] Comprobar: tarjetas con props
- [x] Comprobar: formulario válido
- [x] Comprobar: objeto enviado
- [x] Comprobar: 404

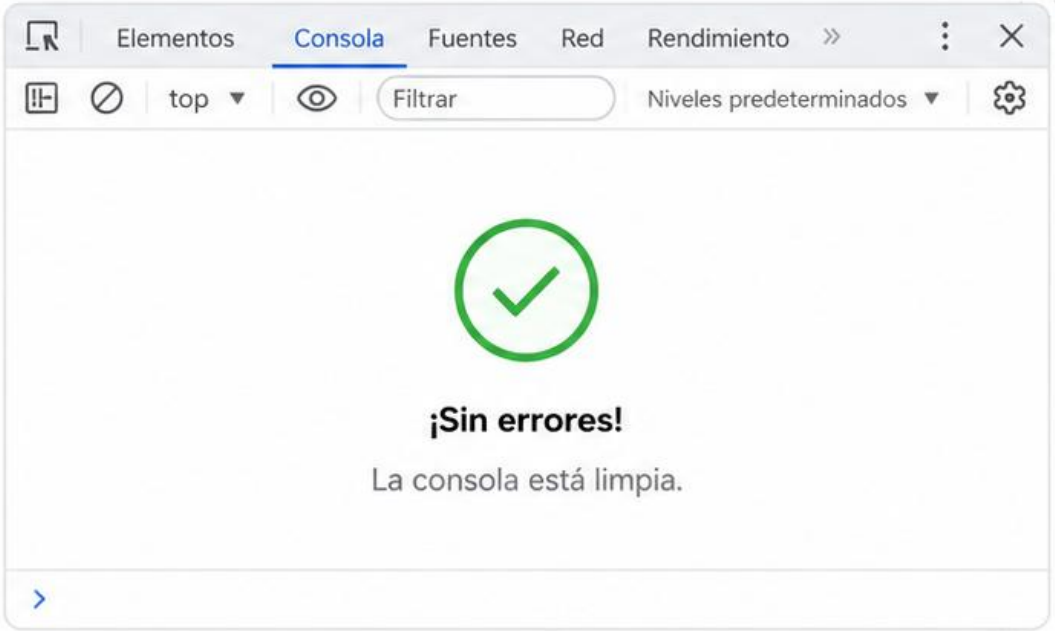
Entregar carpeta src o ZIP según indicación docente.



Autorrevisión

Antes de entregar, verifica cada punto

- [ ] No usé hooks como requisito.
- [ ] No consumí API REST.
- [ ] Cada input tiene name.
- [ ] Las rutas funcionan.
- [ ] EventCard es reutilizable.
- [ ] La 404 existe.
- [ ] No hay errores en consola.



Reconstruimos la arquitectura

Completa primero; las respuestas no se muestran aún

Completa: componente = _____ ;

props = _____ ;

ruta = _____ ;

NavLink = _____ ;

FormData = _____ .

Metacognición

Preguntas: ¿qué componente separarías distinto? ¿qué error de rutas te costó más? ¿por qué FormData permite avanzar sin hooks? ¿qué probarías primero en otra app?

¿qué componente separarías distinto?

¿qué error de rutas te costó más?



¿por qué FormData permite avanzar sin hooks?

¿qué probarías primero en otra app?

Ticket de salida

Actividad breve



Mini-caso: una app tiene /productos y /contacto; ProductCard se repite; contacto necesita nombre y email.

Indica: 1 ruta, 1 componente y 1 regla del formulario.



Próxima semana: Hooks en React.

1. Ruta:
2. Componente:
3. Regla del formulario:



¡MUCHAS GRACIAS POR SU ATENCIÓN!

Universidad Tecnológica del Perú